eNMRIS Database Schema and API Contract

Version: 1.0 **Status:** Agreed design baseline **Date:** 9 November 2026 **Scope:** Prototype medicines register for the Medicines Regulatory Authority (MRA), Eswatini

1. Purpose and design decisions

This contract establishes the data model, approval workflow, and application/FHIR interfaces before feature implementation. It reflects the project plan and the decisions confirmed by the team. `openapi.yaml` is the machine-readable description of the HTTP contract.

- MySQL 8, InnoDB, and `utf8mb4`; application tables use `snake_case`.
- CMS product codes are strings, never numbers, to preserve identifiers exactly.
- The CMS workbook is the source of truth for product-template columns. It contains 19 product rows and 18 unique `programProducts` records. Two trailing rows in that sheet are programme-name notes, not records; the plan's figure of 18 programme links is correct.
- Registration status changes and recall issuance/closure require approval by a different person from the requester.
- If an Admin requests a regulatory change, a specifically authorized Regulatory Officer may approve it. The permission is limited to regulatory approvals and does not grant user or system administration.
- The API separates public, limited medicine verification from authenticated staff and FHIR interfaces.
- The application does not delete regulatory records; corrections and status changes remain traceable.

2. Roles and approval controls

- **Admin:** manages users and configuration; may manage regulatory records; may approve an Officer's regulatory change request.
- **Regulatory Officer:** manages regulatory records but not users or system configuration.
- **Designated approving Officer:** an Officer granted the `approve_regulatory_changes` permission. This Officer may review and approve an Admin's regulatory change request, but gains no administrative privileges.
- **Public visitor:** may search only the approved-medicine and active-recall fields exposed by the public endpoint.
- **FHIR client:** an individually registered external system with read-only scopes and revocable credentials.

The requester and approver must always be different users. Admins may approve Officer requests; designated Officers may approve Admin requests. Approval rights cannot be used to approve the approver's own request. If no independent approver is available, the request remains pending and is escalated to another authorized approver; the self-approval rule is not bypassed.

Every approval or rejection records the requester, approver, reason, decision, and timestamps. The server obtains the actor identity from the authenticated session; clients cannot submit an `approved_by` value.

3. Database schema

3.1 Conventions and constraints

- Use UUID primary keys for application records and UTC timestamps.
- Include `created_at`, `updated_at`, `created_by`, and `updated_by` on mutable business records where applicable. Audit events are append-only and do not have `updated_at`.
- Use foreign keys, unique constraints, checks, and indexes for relationships and lookup fields.
- Apply all writes to a business record, its history, and its audit event in one database transaction.
- Use a unique registration number, subject to confirmation against the MRA's official numbering rules.
- Enforce at most one non-terminal registration for a given product and holder. `PENDING`, `REGISTERED`, and `SUSPENDED` are non-terminal; `REVOKED` and `EXPIRED` are terminal. In MySQL, this can be implemented with a generated nullable active-slot column and a unique index on `(product_id, holder_id, active_slot)`.

- A new registration starts as `PENDING`. Every later status transition requires an approved request. When its expiry date is reached, the system transitions an eligible registration to `EXPIRED` and records a `SYSTEM` history and audit event.

3.2 Tables

- `products` — `id`, unique `product_code` (`VARCHAR`), `dispensable`, `name`, `description`, `pack_size`, `pack_rounding_threshold`, `round_to_zero`, and common timestamps/actors. Preserve the CMS workbook's source values.
- `product_programmes` — `id`, `product_id`, `program`, `doses_per_patient`, `active`, `category`, `full_supply`, `display_order`, `price_per_pack`; unique on `(product_id, program)`. Import all 18 data rows and skip the two explanatory legend rows.
- `organisations` — `id`, `name`, `org_type` (at least `HOLDER`, `MANUFACTURER`), and ISO alpha-3 `country_code`.
- `registrations` — `id`, `product_id`, `holder_id`, unique `registration_number`, `status`, `approval_date`, `expiry_date`, and `status_reason`.
- `registration_status_history` — `id`, `registration_id`, `old_status`, `new_status`, `reason`, `changed_by`, `approved_by` (nullable for system events), and `changed_at`.
- `registration_status_change_requests` — `id`, `registration_id`, `requested_status`, `reason`, `requested_by`, `decision` (`PENDING`, `APPROVED`, `REJECTED`), `decided_by`, `requested_at`, and `decided_at`.
- `batches` — `id`, `product_id`, `batch_code`, `manufacture_date`, and `expiry_date`; unique on `(product_id, batch_code)`.
- `recalls` — `id`, `recall_class` (`I`, `II`, `III`), `reason`, `status` (`ACTIVE`, `CLOSED`), `issued_by`, `issued_at`, `closed_by`, and `closed_at`.
- `recall_batches` — `recall_id`, `batch_id`; supports one recall covering multiple batches and a batch having recall history.
- `recall_change_requests` — `id`, request type (`ISSUE`, `CLOSE`), optional `recall_id`, class/reason/target status as applicable, requester, decision, approver, and timestamps. An associated request-batch table records the batches for a proposed recall.
- `recall_status_history` — `id`, `recall_id`, old/new status, reason, requester, approver, linked change request, and timestamp.
- `app_users` — `id`, `name`, unique `email`, `password_hash`, `role` (`ADMIN`, `OFFICER`), encrypted time-based one-time password (TOTP) secret and MFA state, and `active`.
- `app_user_permissions` — grants scoped permissions such as `approve_regulatory_changes` to designated Officers; record who granted/revoked each permission.
- `app_sessions` — user, hash of the opaque session token, creation/last-activity/expiry times, and revocation time.
- `api_clients`, `api_client_scopes`, and `api_access_tokens` — client identifier, hashed secret, read-only scopes, hashed short-lived bearer tokens, active/expiry/revocation state, and timestamps.
- `audit_log` — append-only event ID, actor type and ID (`USER`, `CLIENT`, or `SYSTEM`), action, entity/table, record ID, before/after JSON, reason, timestamp, and request ID.

The application database account must not have `UPDATE` or `DELETE` permission on `audit_log`. Database administration access must be restricted and audited separately.

4. Application API

Base path: `/api/v1`. All staff routes require authentication and authorization. Mutations validate input on the server and use parameterized SQL.

4.1 Authentication and staff

- `POST /auth/login` — email, password, and time-based one-time password (TOTP) code; creates a secure session.
- `POST /auth/logout` — revokes the current session.
- `GET /auth/me` — returns the authenticated user and effective permissions.
- `GET /approvals?status=pending` — lists regulatory requests the caller is permitted to review.

- `POST /approvals/{requestId}/approve` and `POST /approvals/{requestId}/reject` — record a decision and reason; requester cannot decide their own request.

4.2 Register and regulatory records

- `GET /organisations?q=...` and `POST /organisations` — search/create holders and manufacturers.
- `GET /products?q={term}&page={n}&pageSize={n}` and `GET /products/{id}` — staff product search/read.
- `POST /products/import` and `GET /products/export` — import/export the CMS workbook layout; validate the workbook before committing changes.
- `POST /registrations` — creates a `PENDING` registration and its initial history/audit event.
- `GET /registrations/{id}` and `GET /registrations/{id}/status-history` — read a registration and its history.
- `POST /registrations/{id}/status-change-requests` — body: `{ "targetStatus": "...", "reason": "..." }`.
- `POST /batches` and `GET /batches/{id}` — create/read batches.
- `POST /recall-change-requests` — propose an `ISSUE` with class, reason, and one or more batch IDs.
- `POST /recalls/{id}/closure-requests` — propose closing a recall with a reason.
- Only an approved request changes a registration or recall's effective status. Approval, status update, history, and audit are committed atomically.

4.3 Public verification

- `GET /public/search?q={term}` — no login; returns only limited product details for approved medicines and active recalls. Do not expose staff accounts, private holder data, internal history, or audit details.

4.4 FHIR R4

- `GET /fhir/metadata` — authenticated client capability statement.
- `GET /fhir/Medication?code={cmsProductCode}` and `GET /fhir/Medication/{id}` — authenticated, read-only search/read.
- Authenticate each system independently using revocable client credentials and a short-lived bearer token with read-only FHIR scope.
- Map CMS product code to a documented identifier system; map batches to `Medication.batch`. Document registration authorization and recall mappings; use a documented extension or other suitable R4 representation for recalls, which have no dedicated R4 resource.

4.5 Response conventions

- Success: `{ "data": ..., "meta": ... }`; list endpoints use bounded pagination.
- Error: `{ "error": { "code": "...", "message": "...", "details": ... }, "requestId": "..." }`.
- Use appropriate HTTP status codes. Never return stack traces, SQL, secrets, time-based one-time password (TOTP) values, or password hashes.

5. Security requirements

- **RBAC:** enforce the role and scoped approval permission on every protected route and FHIR scope.
- **MFA:** require a password plus an authenticator-app time-based one-time password (TOTP) for staff in the MVP. This method is not phishing-resistant; passkeys/WebAuthn are recommended before production.
- **Transport:** enforce TLS and HSTS in production. Certificate pinning is not applicable to the current web application; reconsider only if a native mobile app is built.
- **Input/database:** validate request bodies, parameters, uploaded files, and search terms; use parameterized queries exclusively.
- **Login protection:** rate-limit after 5 failed attempts per account/IP in 15 minutes; apply temporary backoff rather than permanent lockout.

- **Sessions:** use a high-entropy opaque session token in a `Secure`, `HttpOnly`, `SameSite` cookie; store only its hash, support revocation, expire after 20 minutes idle, and protect cookie-authenticated mutations against CSRF. Do not bind sessions to browser/device fingerprints.
- **Passwords:** recommended baseline is at least 12 characters, permit passphrases, reject commonly compromised passwords, and do not force periodic changes absent compromise.
- **Audit:** record authentication/security events and all regulatory mutations. Keep the audit log append-only to the application, and restrict direct database access.

6. Contract notes

- The workbook is available as `CMS Products Template.xlsx`; its actual columns are authoritative for import/export. The `programProducts` sheet has 18 data records followed by two legend rows, which are not seed records.
- `database/01_schema.sql` defines the MySQL tables; `database/02_seed.sql` loads the 19 products and 18 programme mappings. `openapi.yaml` describes the application and FHIR endpoints.
- The MRA's registration-number rule and official registration data have not yet been supplied. Keep the numbering format configurable and use test data until approved data is provided.
- Live integration with CMS/eLMIS and mobile applications remain out of scope. The FHIR interface is read-only and designed for future authorized systems.